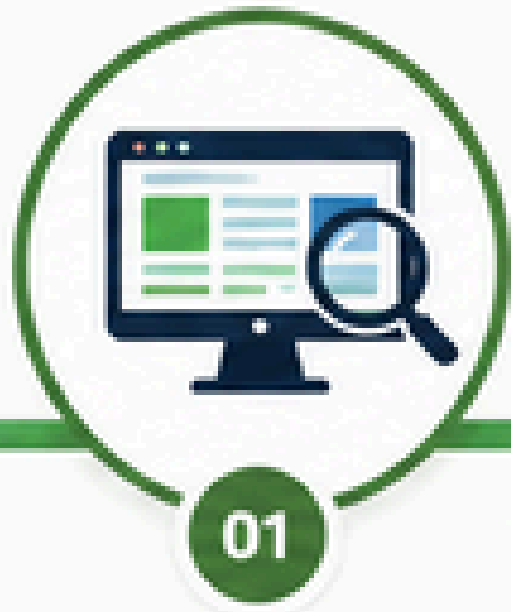


LEARNING OBJECTIVES

Mahasiswa mampu:



01

Memahami
konsep
Web Testing



02

Menjelaskan
Selenium



03

Mengotomatisasi
pengujian web



04

Menulis script
Selenium
sederhana



05

Mengintegrasikan
dengan framework
testing



JUnit

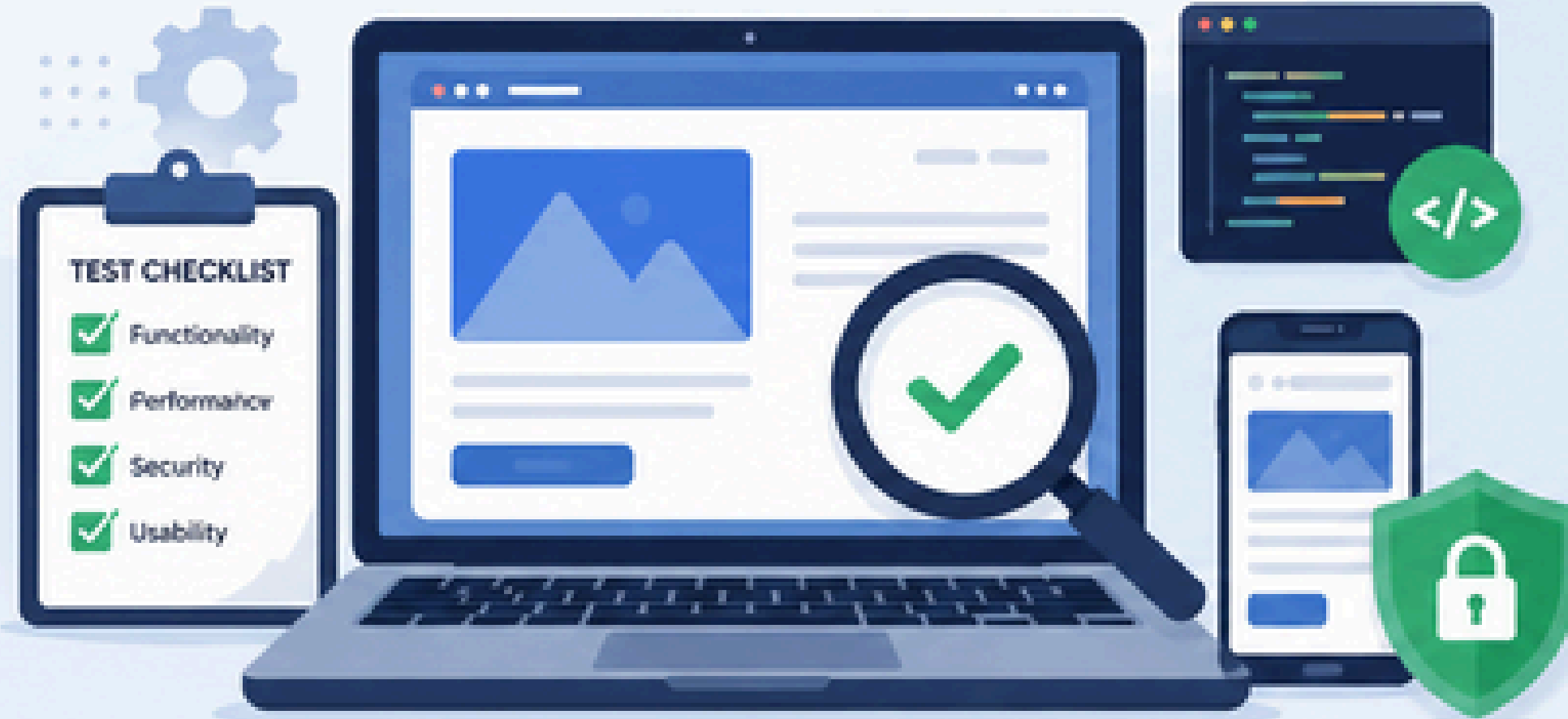
TestNG

maven

APA ITU WEB TESTING?



Web Testing adalah proses pengujian pada aplikasi web untuk memastikan bahwa website berfungsi dengan benar, aman, responsif, dan memberikan pengalaman terbaik bagi pengguna di berbagai kondisi dan perangkat.



Web testing bukan hanya mencari bug, tapi memastikan website Anda **ANDAL, AMAN**, dan **SIAP MEMBERIKAN NILAI TERBAIK**.

KENAPA WEB TESTING ITU PENTING?



MENJAMIN KUALITAS

Memastikan semua fitur bekerja sesuai kebutuhan dan bebas dari bug.



MENINGKATKAN PENGALAMAN PENGGUNA

Website yang stabil, cepat, dan mudah digunakan membuat pengguna lebih puas.



KOMPATIBEL DI SEMUA PERANGKAT & BROWSER

Memastikan tampilan dan fungsi website konsisten di berbagai perangkat, browser, dan ukuran layar.



MENINGKATKAN PERFORMA

Mengidentifikasi masalah performa (*load time, bottleneck*) agar website lebih cepat dan efisien.



MENJAGA KEAMANAN

Mendeteksi celah keamanan lebih awal untuk melindungi data dan mencegah risiko serangan.



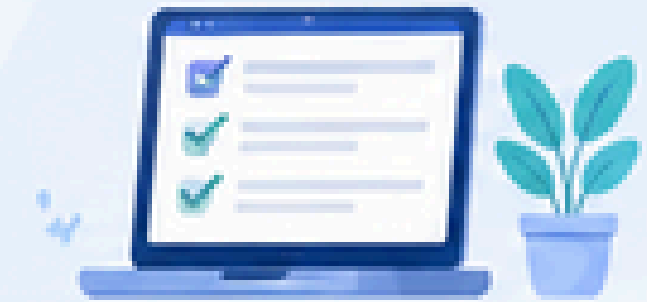
MENGHEMAT WAKTU & BIAYA

Menemukan masalah sejak dini jauh lebih murah daripada memperbaikinya setelah website digunakan banyak orang.



Jenis-Jenis Penggunaan Website untuk Dilakukan Pengujian

Setiap jenis website memiliki tujuan, fitur, dan risiko yang berbeda sehingga membutuhkan pendekatan pengujian yang disesuaikan untuk memastikan kualitas, keamanan, dan performa.

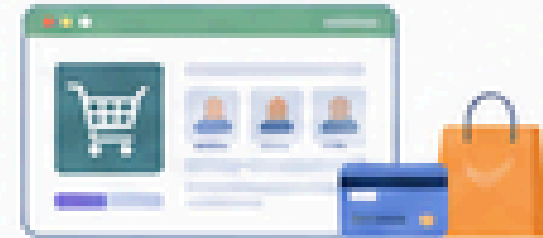


1 Website Perusahaan (Company Profile)



Digunakan untuk memberikan informasi profil, layanan, kontak, dan berita perusahaan. Pengujian fokus pada tampilan, navigasi, form kontak, dan kompatibilitas.

2 Website E-Commerce (Toko Online)



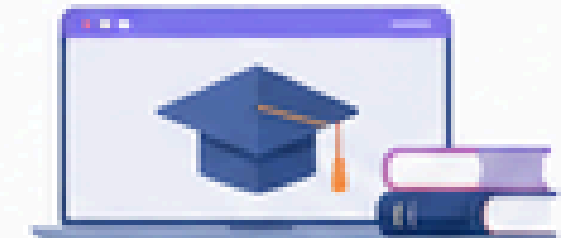
Digunakan untuk transaksi jual beli online. Pengujian meliputi alur belanja, keranjang, pembayaran, stok, kupon, dan keamanan data.

3 Website Portal Berita (News Portal)



Digunakan untuk menyajikan informasi berita/artikel secara update. Pengujian fokus pada performa, pencarian, kategori, dan responsivitas.

4 Website Edukasi / E-Learning (Pembelajaran Online)



Digunakan untuk kegiatan belajar mengajar online. Pengujian mencakup login, materi, kuis, penilaian, forum, dan tracking progres.

5 Website Pemerintahan / Publik (Government)



Digunakan untuk layanan publik dan informasi resmi. Pengujian fokus pada aksesibilitas, keamanan, form layanan, dan kepatuhan.

6 Website Komunitas / Forum (Community)



Digunakan untuk interaksi dan diskusi antar anggota. Pengujian meliputi registrasi, membuat topik, reply, notifikasi, dan moderasi.

7 Website Web Application (Sistem Informasi)



Digunakan untuk proses bisnis dan pengelolaan data. Pengujian mencakup fungsionalitas modul, integrasi, validasi data, dan keamanan.

8 Website Landing Page (Kampanye / Marketing)



Digunakan untuk kampanye/promosi produk atau layanan. Pengujian fokus pada CTA, form lead, tracking, dan performa.



METODE PENGUJIAN WEBSITE

BERDASARKAN JENISNYA



Metode pengujian website adalah teknik atau cara yang digunakan untuk memastikan website berfungsi sesuai kebutuhan, aman, dan memberikan pengalaman terbaik bagi pengguna.



1. FUNCTIONAL TESTING

Menguji apakah semua fungsi di website berjalan sesuai dengan kebutuhan dan spesifikasi.

Contoh yang diuji:

- Form login
- Pencarian
- Tombol & link
- Proses checkout



2. USABILITY TESTING

Menguji kemudahan penggunaan website dari sudut pandang pengguna.

Contoh yang diuji:

- Navigasi mudah dipahami
- Tata letak (layout)
- Kejelasan konten
- Pengalaman pengguna secara keseluruhan



3. PERFORMANCE TESTING

Menguji kecepatan, stabilitas, dan kemampuan website dalam menahan beban pengguna.

Contoh yang diuji:

- Waktu muat halaman
- Respons server
- Uji beban (load testing)
- Stabilitas saat trafik tinggi



4. SECURITY TESTING

Menguji keamanan website dari kerentanan dan ancaman yang dapat membahayakan data pengguna.

Contoh yang diuji:

- SQL Injection
- Cross Site Scripting (XSS)
- Autentikasi & otorisasi
- Keamanan data pengguna



5. COMPATIBILITY TESTING

Menguji kompatibilitas website pada berbagai perangkat, browser, dan sistem operasi.

Contoh yang diuji:

- Browser (Chrome, Firefox, Edge, Safari)
- Perangkat (Desktop, Tablet, Mobile)
- Sistem Operasi (Windows, macOS, Android, iOS)



6. MAINTAINABILITY TESTING

Menguji struktur dan kualitas kode agar website mudah dipelihara dan dikembangkan.

Contoh yang diuji:

- Keterbacaan kode
- Struktur kode
- Dokumentasi
- Kemudahan modifikasi



Tujuan Utama

Memastikan website berkualitas, aman, cepat, mudah digunakan, dan sesuai kebutuhan pengguna.



Apa itu Selenium?

Selenium adalah **framework open-source** untuk otomatisasi browser.



- Open-source
- Cross-browser
- Multi-language



```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://example.com")
driver.find_element(By.ID, "username")
    .send_keys("admin")
driver.find_element(By.ID, "password")
    .send_keys("123456")
driver.find_element(By.ID, "loginBtn")
    .click()
```



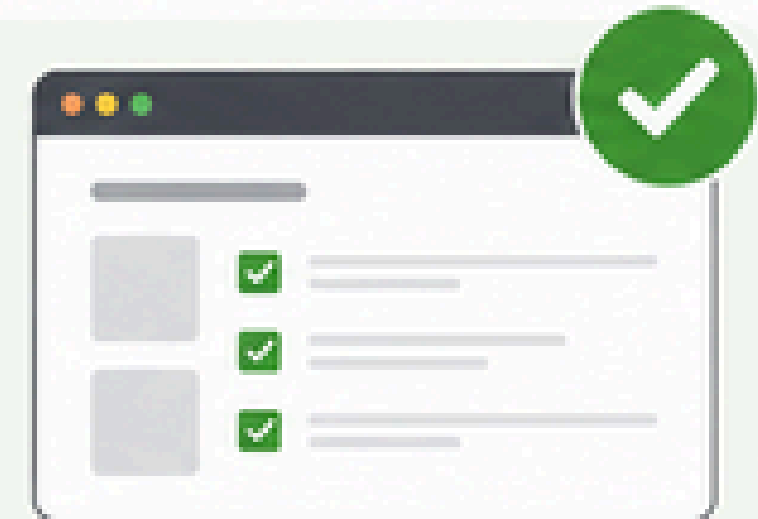
Open-source & Cross-platform

Digunakan untuk:



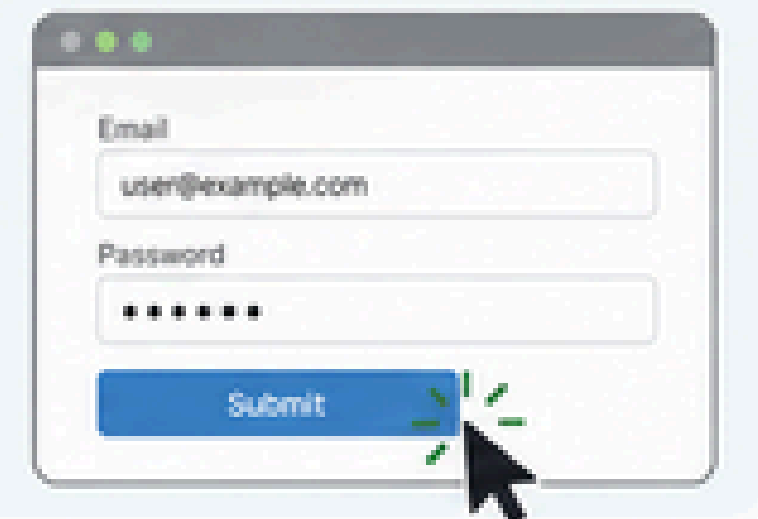
1. Testing web otomatis

Menjalankan test case secara otomatis untuk memastikan fitur website bekerja dengan baik.



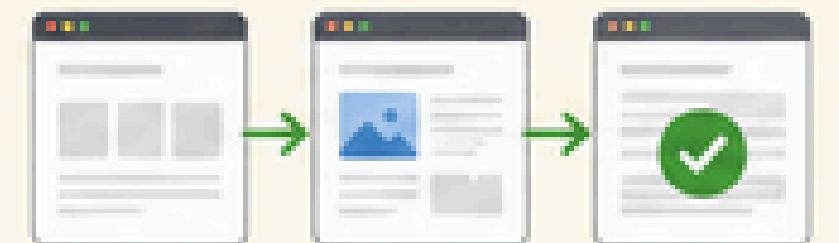
2. Simulasi user (klik, input, navigasi)

Meniru interaksi pengguna seperti klik tombol, mengisi form, scroll, dropdown, dll.



3. Navigasi otomatis

Berpindah halaman, menangani pop-up, menunggu elemen, dan banyak lagi.





LANGKAH-LANGKAH MENGGUNAKAN SELENIUM LENGKAP

1 PERSIAPAN



- Install Java (JDK)
- Install IDE (Eclipse/IntelliJ)
- Install Browser



2 BUAT PROJECT



Buat project baru
(Maven/Gradle/Java Project)

Contoh struktur Maven:

```
my-selenium-project/  
├── src/  
│   ├── test/  
│   └── java/  
└── pom.xml
```

3 TAMBAHKAN DEPENDENCY



Tambahkan dependency Selenium
di file pom.xml

```
<dependencies>  
  <dependency>  
    <groupId>org.seleniumhq.selenium</groupId>  
    <artifactId>selenium-java</artifactId>  
    <version>4.20.0</version>  
  </dependency>  
</dependencies>
```

4 DOWNLOAD WEBDRIVER



- Download WebDriver sesuai browser (ChromeDriver, GeckoDriver, etc)
- Pastikan versi driver kompatibel dengan browser
- Letakkan file driver di folder yang mudah diakses

5 BUAT PROGRAM PERTAMA

Buat kelas Java dan tulis script pertama

```
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.chrome.ChromeDriver;  
  
public class FirstTest {  
    public static void main(String[] args) {  
        WebDriver driver = new ChromeDriver();  
        driver.get("https://www.google.com");  
        System.out.println("Selenium berjalan!");  
        driver.quit();  
    }  
}
```

6 MENJALANKAN TEST



Klik kanan pada file/kelas
→ Run As → Java Application
atau gunakan Maven:

```
mvn test
```

7 FIND ELEMENT

Temukan element pada halaman
menggunakan locator

```
driver.findElement(By.id("id"));  
driver.findElement(By.name("name"));  
driver.findElement(By.className("class"));  
driver.findElement(By.cssSelector("css"));  
driver.findElement(By.xpath("xpath"));
```

8 INTERAKSI DENGAN ELEMENT

Lakukan aksi pada element

```
WebElement element = driver.findElement(By.id("q"));  
element.sendKeys("Selenium WebDriver");  
element.click();  
element.clear();  
String text = element.getText();  
String value = element.getAttribute("value");
```

9 WAIT (MENUNGGU)

Gunakan wait untuk menunggu element
muncul/siap

```
WebDriverWait wait = new WebDriverWait(driver,  
    Duration.ofSeconds(10));  
WebElement element = wait.until(  
    ExpectedConditions.visibilityOfElementLocated  
        (By.id("example"))  
);
```

10 SCREENSHOT (OPSIONAL)

Ambil screenshot saat dibutuhkan

```
TakesScreenshot ts = (TakesScreenshot) driver;  
File src = ts.getScreenshotAs(OutputType.FILE);  
Files.copy(src.toPath(), Paths.get("screenshot.png"),  
    StandardCopyOption.REPLACE_EXISTING);
```

11 HANDLE ALERT

Menangani alert/konfirmasi/prompt

```
Alert alert = driver.switchTo().alert();  
alert.accept(); // OK  
alert.dismiss(); // Cancel  
alert.sendKeys("teks"); // Prompt
```

12 HANDLE WINDOW/TAB

Berpindah antar window atau tab

```
String parent = driver.getWindowHandle();  
for (String handle : driver.getWindowHandles())  
    driver.switchTo().window(handle);  
driver.switchTo().window(parent);
```

13 DATA DRIVEN (OPSIONAL)

Gunakan data dari file (Excel/CSV/JSON)
untuk membuat test lebih fleksibel



Excel



CSV



JSON

14 FRAME/I FRAME

Berpindah ke frame/frame

```
driver.switchTo().frame("frameName");  
// atau  
WebElement frame = driver.findElement(By.id("frameId"));  
driver.switchTo().frame(frame);  
driver.switchTo().defaultContent();
```

15 TUTUP BROWSER

Tutup browser setelah test selesai

```
driver.quit(); // Menutup seluruh jendela  
driver.close(); // Menutup jendela aktif
```



TIPS



Selalu gunakan wait yang tepat



Hindari hard sleep



Gunakan locator yang stabil



Jaga test tetap independen



Bersihkan data setelah test



SELESAI!

Automation Test siap digunakan

BrowserStack Cloud Testing

Test website di berbagai browser & device
Automation **tanpa setup lokal**

Platform cloud untuk pengujian otomatis lintas browser
dan device tanpa konfigurasi tambahan.

KELEBIHAN



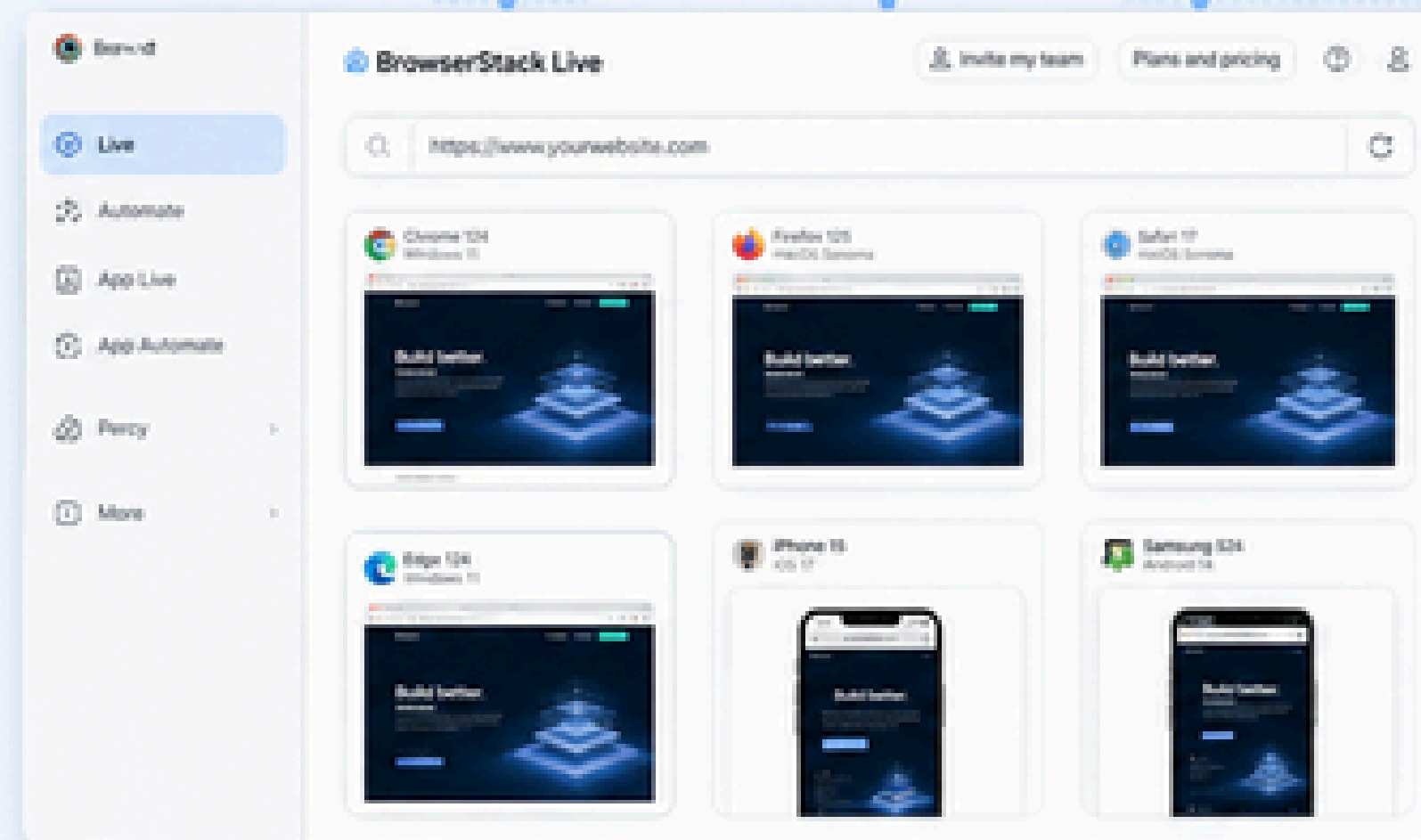
Tidak perlu install
Semua berjalan di cloud, tidak perlu
install atau konfigurasi lokal.



Support banyak browser
Dukung versi terbaru dan versi lama
dari banyak browser populer.



Real device testing
Uji di real device fisik untuk hasil yang
lebih akurat.

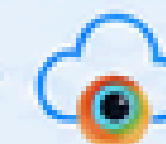


```
Selenium Playwright Cypress Appium Puppeteer  
1 import { remote } from 'selenium-webdriver';  
2 import { BrowserStackOptions } from 'browserstack-local';  
3  
4 let driver = await new Builder()  
5   .usingServer('https://hub-cloud.browserstack.com/webhub')  
6   .withCapabilities({  
7     'browserName': 'Chrome',  
8     'browserVersion': '124.0',  
9     'os': 'Windows',  
10    'osVersion': '11',  
11    'name': 'Stack Cloud Test'  
12  })  
13  .build();
```



Tulis Test

Buat test script
menggunakan framework
favorit Anda.



Jalankan di Cloud

Test dijalankan di infrastruktur
BrowserStack tanpa perlu
setup apapun.



Hasil Real-time

Lihat hasil test secara real-time
di berbagai browser & device
dengan detail lengkap.

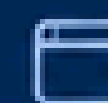


Lebih cepat • Lebih mudah • Lebih luas jangkauannya

BrowserStack membantu tim QA & Developer merilis produk berkualitas dengan percaya diri.



3000+
Real Devices



20+
Browsers



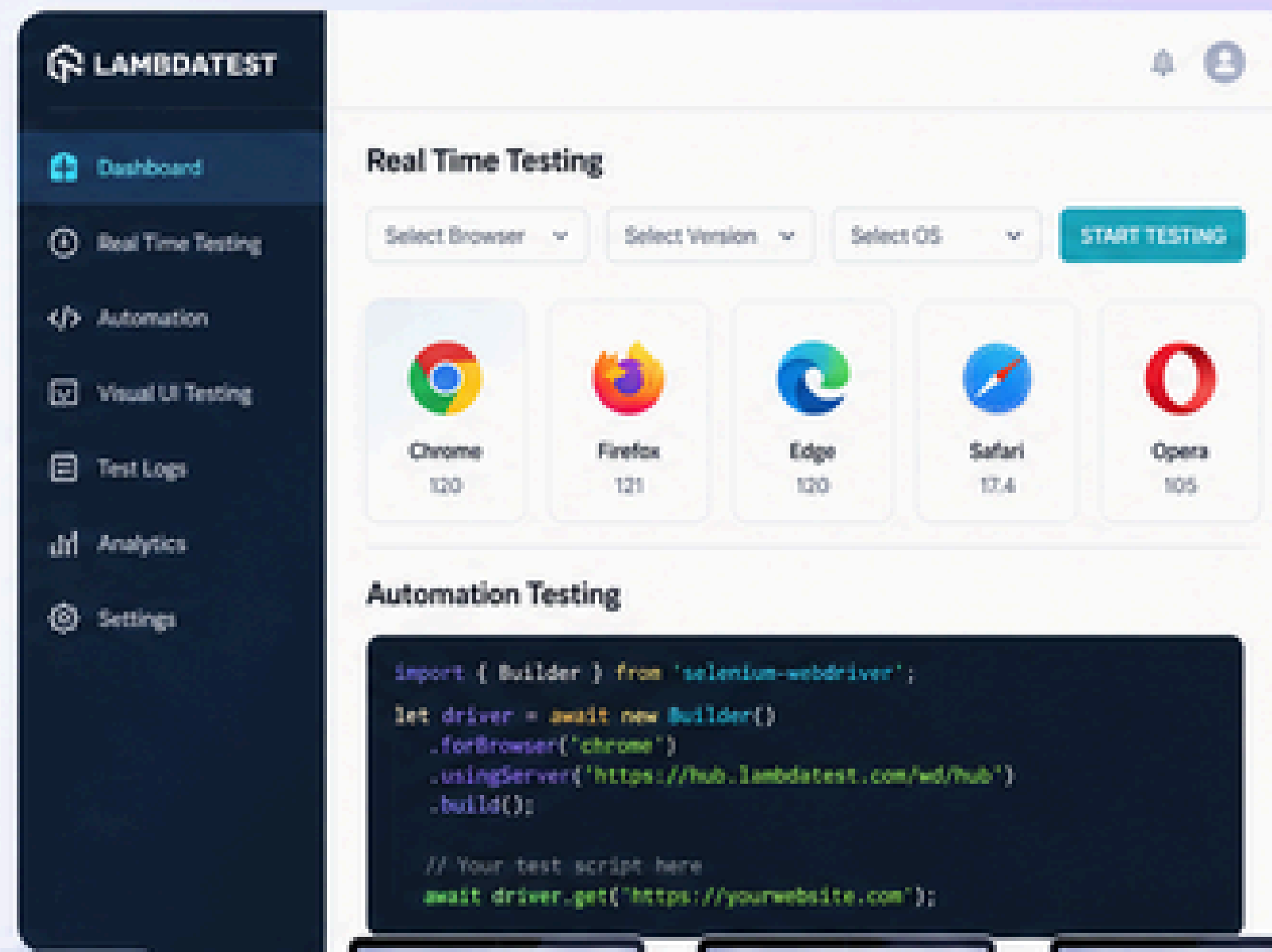
99.9%
Uptime



ONLINE • AUTOMATION • CLOUD

Cross-browser testing dan Automation testing di Cloud

Uji web/aplikasi Anda di 3000+ browser & OS nyata dengan Automation di Cloud.



3000+
Browser & OS
Nyata



COCOK UNTUK MAHASISWA

Tidak perlu setup environment,
langsung testing di cloud!

- ✓ Tanpa instalasi ribet
- ✓ Hemat waktu & biaya
- ✓ Akses kapan saja, di mana saja



lambdatest.com



3000+
Browser & OS
Nyata



100%
Berbasis Cloud



Fleksibel
dan Scalable



Aman &
Terpercaya

AI Testing Tool Smarter. Faster. Better.

AI-powered test automation
that works for everyone



Self-healing Test

Test otomatis menyesuaikan diri saat ada perubahan UI, sehingga lebih stabil dan minim maintenance.



Mudah Digunakan

Buat, jalankan, dan maintain test dengan mudah. Tanpa coding!



AI-Powered
Intelligent
Automation



No Coding
Create tests
visually



Self-Healing
Tests adapt to
UI changes



Faster Releases
High quality,
less effort



For Everyone
QA, Dev, or BA
— all in one tool



Test



Project



Tests



Suites



Runs



Analytics



Settings

< E-commerce Checkout Flow

▶ Run



TA

Tests

Search tests

- 1 Navigate to <https://shop.demo.com>
- 2 Click "Add to Cart"
- 3 Click "Checkout"
- 4 Set Text "Name" John Doe
- 5 Set Text "Email" john@email.com
- 6 Click "Place Order"
- 7 Validate Element "Thank you"

+ Add Step

Last Run Results



✓ 7/7 Steps Passed

⌚ Duration: 00:01:24

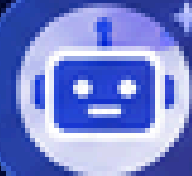
💻 Environment: Chrome | Windows 11



Self-Healing

Test updated
automatically

✓ No action needed



AI makes testing smarter.
You ship with **confidence**.



Super mudah – no code

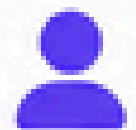
Cukup klik,
test case jadi otomatis!

Test Otomatis Tanpa Coding!

Record & replay testing
Tanpa coding sama sekali



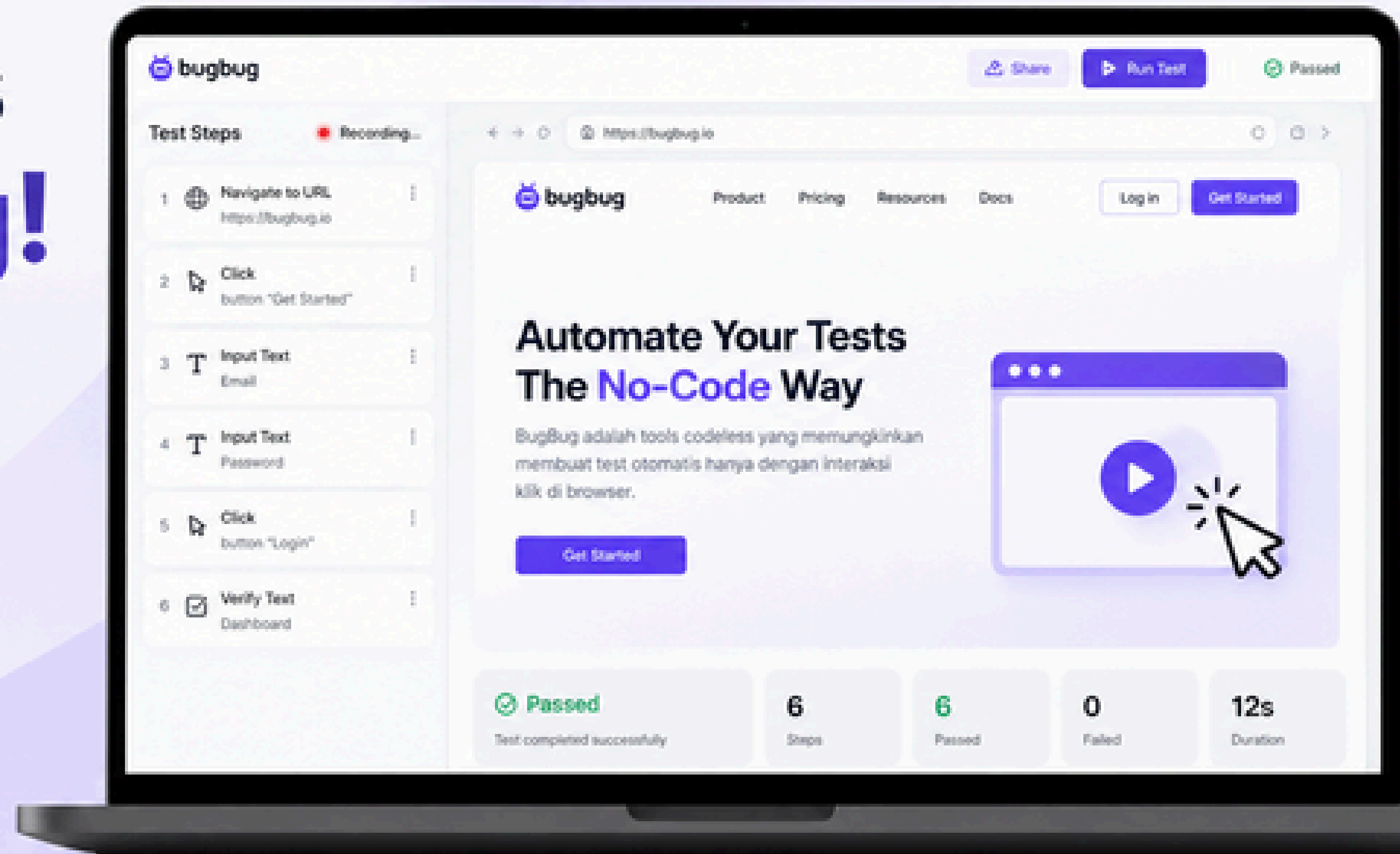
Website:
bugbug.io



Sangat cocok
untuk pemula



Bisa langsung
dari browser



BugBug adalah tools codeless yang memungkinkan
membuat test otomatis hanya dengan interaksi klik di browser



Record
Interaksi



Replay
Otomatis



Hasil
Akurat



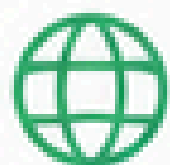
Katalon PLATFORM



Online Platform
Access Anywhere, Anytime

Low-code + Online Test Automation Platform

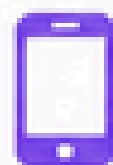
Web, API, mobile testing | Automation + reporting



Web Testing



API Testing



Mobile Testing

Kelebihan:



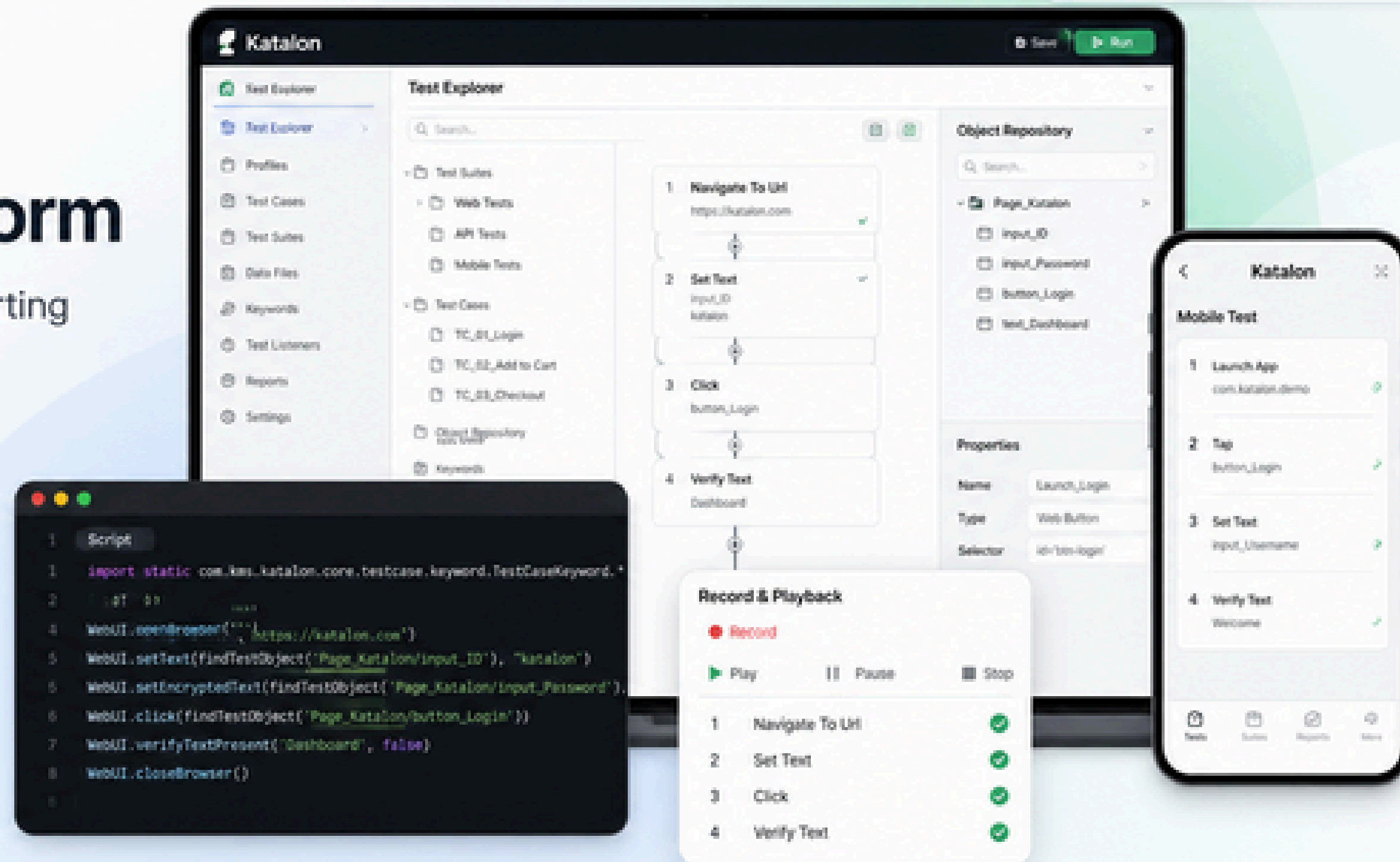
Ada mode tanpa coding

Buat test otomatis dengan mudah menggunakan record & playback.



Bisa automation kompleks

Gunakan scripting untuk kebutuhan testing yang lebih fleksibel dan kompleks.



Katalon menyediakan fitur - **record & playback** + **scripting** untuk berbagai jenis testing



Fleksibel
Untuk semua level QA



Lengkap
Semua jenis testing



Insightful
Reporting yang informatif



Online
Aman & selalu up-to-date



katalon.com

Mana yang Paling Cocok untuk Mahasiswa?

Pilih tools automation testing sesuai level kamu dan tingkat kenyamanan dengan coding.

Level	Rekomendasi	
 Pemula banget Baru mulai belajar automation testing	 BugBug	 Selenium IDE
 Tanpa coding Ingin praktis tanpa menulis kode	 Testim	 BugBug
 Semi advanced Sudah paham dasar coding, ingin lebih fleksibel	 Katalon	
 Cloud testing Ingin testing di banyak device & browser di cloud	 BrowserStack	 LambdaTest

★ Tools hanya alat, pemahaman konsep adalah kunci!



Tips untuk Mahasiswa

- ✓ Mulai dari yang mudah dulu
- ✓ Pahami konsep dasarnya
- ✓ Konsisten & banyak praktik!





Thank You!

See You Next Week